28

BÖLÜM: THREAD HAVUZLARI, EXECUTOR'LAR & WORK-STEALING | MODULE: THREAD POOLS, EXECUTORS & WORK-STEALING ALGORITHMS

ScheduledThreadPoolExecutor: Zamanlanmış Görevler, Drift & İptal Mekanizmaları

ScheduledThreadPoolExecutor: Timer Scheduling, Delay Drift & Task Cancellation

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────┐
│              SCHEDULEDTHREADPOOLEXECUTOR GÖREV TÜRLERİ   │
└──────────────────────────────────────────────────────────┘

 1. scheduleAtFixedRate (Sabit Hızlı Frekans):
    Periyot: 5 saniye | Görev süresi: 2 saniye
    [ Görev 1 (2s) ] ──3s bekle──▶ [ Görev 2 (2s) ] ──3s bekle──▶ [ Görev 3 ]
    ⚠ Eğer görev 7 saniye sürerse? Sonraki görev GECİKMELİ olarak anında başlar!

 2. scheduleWithFixedDelay (Sabit Aralık Molası):
    Gecikme: 5 saniye | Görev süresi: 2 saniye
    [ Görev 1 (2s) ] ──TAM 5 SANİYE BEKLE──▶ [ Görev 2 (2s) ] ──TAM 5s BEKLE──▶
    İş ne kadar sürerse sürsün, iki işin bitişi ile başlangıcı arası DAİMA 5 saniyedir!

 İÇ MİMARİ: `DelayedWorkQueue` (Min-Heap / Öncelik Kuyruğu)
 En yakın sürede çalışacak görev kuyruğun tepesindedir (O(log N) ekleme).
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

İki tip ilaç saati gibi düşünün: 1) FixedRate (Sabit Saat): Doktor 'Her sabah saat tam 08:00'de ilaç iç' dedi. Kahvaltıyı uzatsanız bile saat 08:00 olunca ilacı almak zorundasınız. 2) FixedDelay (Sabit Mola): Doktor 'Yemeğin bittikten TAM 2 saat sonra ilacını iç' dedi. Kahvaltı ister 10 dakika ister 1 saat sürsün; kahvaltı bittiği andan itibaren kronometre 2 saat saymaya başlar! Eğer bir işin önceki iş bittikten sonra belirli bir süre dinlenmesini istiyorsanız `scheduleWithFixedDelay` seçmelisiniz!

🇬🇧 ENGLISH (ELI5)

Compare two medicine schedules: 1) `scheduleAtFixedRate`: 'Take medicine at 08:00, 12:00, 16:00'. If lunch runs late, the schedule still demands delivery at the clock strike. 2) `scheduleWithFixedDelay`: 'Take medicine exactly 4 hours AFTER finishing your previous meal'. The timer starts only when the prior task completes. For periodic database synchronization or cleanup jobs, `scheduleWithFixedDelay` prevents overlapping runs if a task takes longer than expected!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ScheduledThreadPoolExecutor İç Mimarisi ve İstisna Tuzağı: 1) DelayedWorkQueue (Min-Heap): Görevler `ScheduledFutureTask` nesnelerine sarılır. Kuyruk, çalışma zamanına göre sıralanmış bir ikili yığın (Binary Min-Heap) tutar. 2) Sessiz Ölüm Tuzağı (Silent Death): Periyodik çalışan bir görev (`Runnable`) içinde yakalanmamış bir `RuntimeException` fırlarsa, `ScheduledExecutorService` o görevi SESSİZCE İPTAL EDER ve bir daha asla çalıştırmaz! Konsola hiçbir hata basılmaz! Görev arka planda ölmüştür ve kimsenin haberi olmaz. 3) Çözüm: Periyodik görevlerin `run()` metodu en dışta `try-catch(Throwable t)` ile sarılmalıdır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

ScheduledExecutor Deep-Dive & The Silent Death Hazard: 1) DelayedWorkQueue Topology: Backed by a concurrent Fibonacci/Min-Heap, ordering scheduled tasks by trigger epoch (`getDelay(TimeUnit.NANOSECONDS)`). 2) FixedRate vs FixedDelay Under Overrun: If a task under `scheduleAtFixedRate` exceeds the interval, subsequent executions do not overlap (single-thread constraint per task instance), but they stack up without pause, causing high CPU storms. `scheduleWithFixedDelay` strictly guarantees `delay` elapsed time between execution completion and next launch. 3) ⚠️ The Fatal Silent Death Trap: If an unhandled `Throwable` escapes a scheduled Runnable, the executor halts all future executions of that task permanently, suppressing the stack trace! Always envelop scheduled loops with `try-catch (Throwable t)`.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Güvenli Periyodik Görev Zamanlayıcı
import java.util.concurrent.*;

public class ResilientScheduler {
    void main() {
        ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(2);

        // Güvenli Periyodik Görev (FixedDelay)
        scheduler.scheduleWithFixedDelay(() -> {
            try {
                IO.println("Cache temizliği başladı: " + System.currentTimeMillis());
                Thread.sleep(100);
            } catch (Throwable t) {
                // ⚠ KESİNLİKLE THROWABLE YAKALANMALIDIR! Aksi halde scheduler görevi öldürür!
                IO.println("Hata yakalandı, görev ölmedi: " + t.getMessage());
            }
        }, 0, 1, TimeUnit.SECONDS);

        // 3 saniye sonra durdur
        scheduler.schedule(() -> {
            IO.println("Scheduler kapatılıyor...");
            scheduler.shutdown();
        }, 3, TimeUnit.SECONDS);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Üretim Kabusu: 'ScheduledExecutorService'e attığım cron görevi 3 gün çalıştıktan sonra durdu, hata da vermedi, neden?' Yanıt: 'Görev içinde bir NullPointerException fırlamıştır ve catch edilmemiştir. Scheduler görevi kalıcı olarak iptal etmiştir!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Real-World Production Horror: 'Our periodic cache refresher ran for 4 days then mysteriously vanished without crashing the JVM. Why?' Answer: 'An uncaught NullPointerException escaped the Runnable. ScheduledExecutorService silently terminated future iterations without logging to stdout.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- scheduleAtFixedRate sabit frekansla çalışır; scheduleWithFixedDelay iş bittikten sonra bekler.
- Yakalanamayan RuntimeException periyodik görevleri SESSİZCE ÖLDÜRÜR.
- Periyodik görevlerin gövdesi daima try-catch(Throwable) ile sarılmalıdır.

🎯 Key Takeaways for the Interview

- scheduleAtFixedRate targets fixed intervals; scheduleWithFixedDelay enforces delay between runs.
- Uncaught exceptions silently cancel future executions of scheduled tasks permanently.
- Always wrap scheduled task bodies inside a defensive try-catch(Throwable) block.